Informe de Laboratorio 2: Pilas y Filas

Listas Enlazadas vs. Arreglos Redimensionables en un Rastreador Web Offline

Integrantes:
Santiago González
Sebastián Mamani
Carlos García

29 de octubre de 2026

Resumen

Este documento presenta la evaluación teórico-empírica de las estructuras de datos Pila y Fila en sus versiones enlazadas (`PilaEnlazada`, `FilaEnlazada`) y basadas en arreglos dinámicos (`PilaArreglo`, `FilaArreglo`), adaptadas desde la biblioteca `algs4`. El rendimiento se evalúa en un contexto aplicado de rastreo web offline (`WebCrawlerOffline`), utilizando una pila para la validación LIFO de etiquetas y una fila para el recorrido BFS del sitio web generado por `GeneradorWeb`. Las pruebas se midieron sobre tamaños $N \in [1024, 262144]$ con 30 repeticiones controladas por semilla (*seed*). Los resultados se procesaron en la plantilla `analisis.xlsx` para validar la complejidad lineal amortizada $O(N)$ mediante ratios de doblado $T(2N)/T(N)$ y exponentes $\log_2$.

Índice

1. Introducción **3**
1.1. Contexto y Motivación . . . 3
1.2. Planteamiento del Problema . . . 3
1.3. Objetivos . . . 3
1.3.1. Objetivo General . . . 3
1.3.2. Objetivos Específicos . . . 3

2. Arquitectura y Diseño de Software **3**
2.1. Adaptación e Instrumentación de Clases (`algs4`) . . . 3
2.2. Análisis Teórico de Operaciones Individuales (Pregunta 1) . . . 4
2.3. Diseño en PlantUML . . . 4

3. Metodología Experimental **4**
3.1. Dominio de Aplicación y Protocolo . . . 4
3.2. Análisis del Orden de Crecimiento Teórico (Pregunta 2) . . . 4
3.3. Aislamiento y Alternancia de Medición . . . 5

4. Análisis de Resultados **5**
4.1. Predicción vs. Evidencia Empírica (Pregunta 3) . . . 5
4.2. Factores Constantes y Rendimiento Práctico (Pregunta 4) . . . 7
4.3. Comportamiento del Redimensionamiento (Pregunta 5) . . . 7

5. Conclusiones y Discusión Teórica **7**
5.1. Análisis Teórico de Uso de Memoria (Pregunta 6) . . . 7
5.2. Conclusiones Generales . . . 7

1. Introducción

1.1. Contexto y Motivación

Las estructuras de datos abstractas Pila y Fila separan su contrato operacional de su representación interna en memoria. En este laboratorio se analizan dos implementaciones clásicas:

1. **Listas enlazadas (Linked Lists):** Asignación dinámica e individual de nodos vinculados mediante referencias.
2. **Arreglos dinámicos (Resizing Arrays):** Espacio de memoria contiguo que duplica o reduce su capacidad dinámicamente.

Estas estructuras se aplican en el dominio de un rastreador web *offline*: una fila mantiene la frontera de páginas pendientes en orden FIFO (búsqueda en anchura / BFS), mientras que una pila valida el cierre correcto de etiquetas HTML en orden LIFO.

1.2. Planteamiento del Problema

Aunque ambas representaciones ofrecen un costo constante teórico $O(1)$ (puro o amortizado), factores como la localidad de referencia en caché, las pausas por redimensionamiento de arreglos y la sobrecarga espacial de punteros generan diferencias operativas en ejecuciones reales. Es necesario evaluar el impacto de estas diferencias en el tiempo de procesamiento y el uso de memoria a medida que el tamaño N se duplica.

1.3. Objetivos

1.3.1. Objetivo General

Comparar teórica y empíricamente el rendimiento de implementaciones enlazadas y de arreglo dinámico para los TDA Pila y Fila mediante la metodología de duplicación de instancia ($T(2N)/T(N)$) sobre las operaciones de un rastreador web.

1.3.2. Objetivos Específicos

- Adaptar e instrumentar las clases base de `algs4` (`Stack`, `ResizingArrayStack`, `Queue`, `ResizingArrayQueue`).
- Implementar los componentes `WebCrawlerOffline` y `GeneradorWeb` aislando los tiempos de generación de datos de los bloques de medición.
- Analizar estadísticamente las métricas de promedio, desviación estándar, coeficiente de variación (CV), ratios $T(2N)/T(N)$ y $\log_2$ mediante `analisis.xlsx`.

2. Arquitectura y Diseño de Software

2.1. Adaptación e Instrumentación de Clases (`algs4`)

Se implementaron las interfaces `Pila<T>` y `Fila<T>`, adaptando fielmente las clases originales de Sedgewick& Wayne. Las clases `PilaArreglo<T>` y `FilaArreglo<T>` incorporan

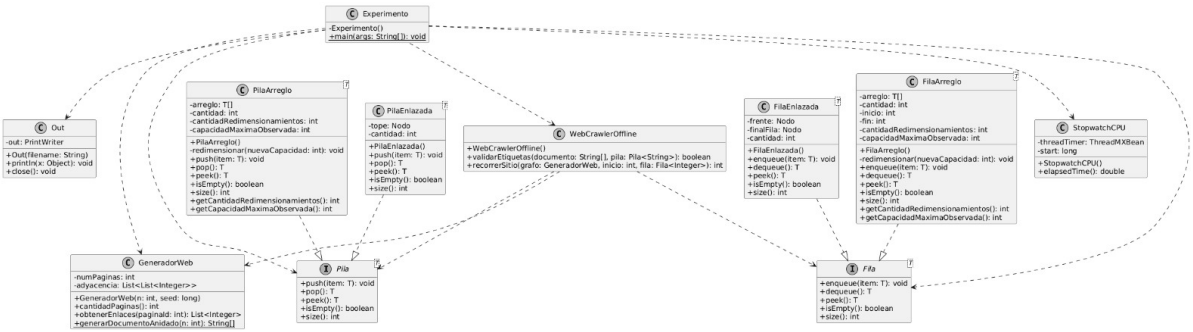
métricas diagnósticas mediante `getCantidadRedimensionamientos()` y `getCapacidad Maxima()`.

2.2. Análisis Teórico de Operaciones Individuales (Pregunta 1)

- **Representación Enlazada:** Operaciones `push`, `pop`, `enqueue` y `dequeue` tienen complejidad $O(1)$ en el **peor caso** (creación o eliminación directa de nodos sin reestructuración).
- **Representación Arreglo Dinámico:** Tienen un costo de $O(N)$ en el **peor caso** cuando se desencadena un redimensionamiento (copiado de elementos), pero un costo $O(1)$ **amortizado** a lo largo de una secuencia de operaciones.

2.3. Diseño en PlantUML

A contuniacion. Se presenta la arquitectura general del sistema. Se destacan las interfaces genéricas `Pila<T>` y `Fila<T>`, las cuales independizan el contrato operacional de sus dos implementaciones concretas (basadas en arreglos dinámicos y listas enlazadas). La clase `WebCrawlerOffline` consume estas interfaces de forma transparente para realizar la validación LIFO de etiquetas y el recorrido BFS del sitio. Finalmente, `Experimento` coordina la ejecución midiendo los tiempos mediante `StopwatchCPU` y registrando los microdatos con `Out`.



3. Metodología Experimental

3.1. Dominio de Aplicación y Protocolo

- **Experimento 1 (Pila):** Evaluación del método `validarEtiquetas(tokens, pila)` sobre documentos sintéticos generados por `GeneradorWeb.generarDocumentoAnidado(n)`.
- **Experimento 2 (Fila):** Recorrido de grafos web mediante `recorrerSitio(sitio, 0, frontera)` (BFS) explorando n páginas y $4n - 1$ enlaces.

3.2. Análisis del Orden de Crecimiento Teórico (Pregunta 2)

- `validarEtiquetas` procesa $2n$ tokens (n aperturas y n cierres), realizando $2n$ operaciones sobre la pila, resultando en complejidad total lineal $O(n)$.
- `recorrerSitio` procesa n páginas y examina $4n - 1$ enlaces. Como cada página entra y sale a lo sumo una vez de la fila, el trabajo total es $O(n)$.

3.3. Aislamiento y Alternancia de Medición

Se ejecutaron 30 repeticiones por cada $n = 2^t$ ($t \in [10, 18]$). Se incluyó calentamiento previo y alternancia de orden de medición en iteraciones pares e impares para eliminar sesgos de ejecución.

4. Análisis de Resultados

4.1. Predicción vs. Evidencia Empírica (Pregunta 3)

Al analizar procesos de complejidad lineal $O(N)$, la teoría del método de duplicación predice que al duplicar el tamaño de la entrada ($2N$), el tiempo de ejecución se duplicará ($T(2N)/T(N) \to 2$), lo que corresponde a un exponente $\log_2(T(2N)/T(N)) \to 1$.

En la Figura 1 y la Figura 2 se presentan las mediciones empíricas en escala Log-Log para Pila y Fila respectivamente. La tendencia lineal observada en ambas gráficas confirma empíricamente la complejidad lineal de los algoritmos de validación de etiquetas y recorrido del sitio web, validando que las operaciones individuales tienen un costo promedio constante $O(1)$.

Cabe destacar que para tamaños pequeños ($N \leq 8192$), se observan fluctuaciones con valores de $\log_2$(Ratio) menores a 1 o negativos. Esto no contradice la teoría, sino que responde al ruido experimental del sistema operativo y latencias de CPU en mediciones del orden de microsegundos.

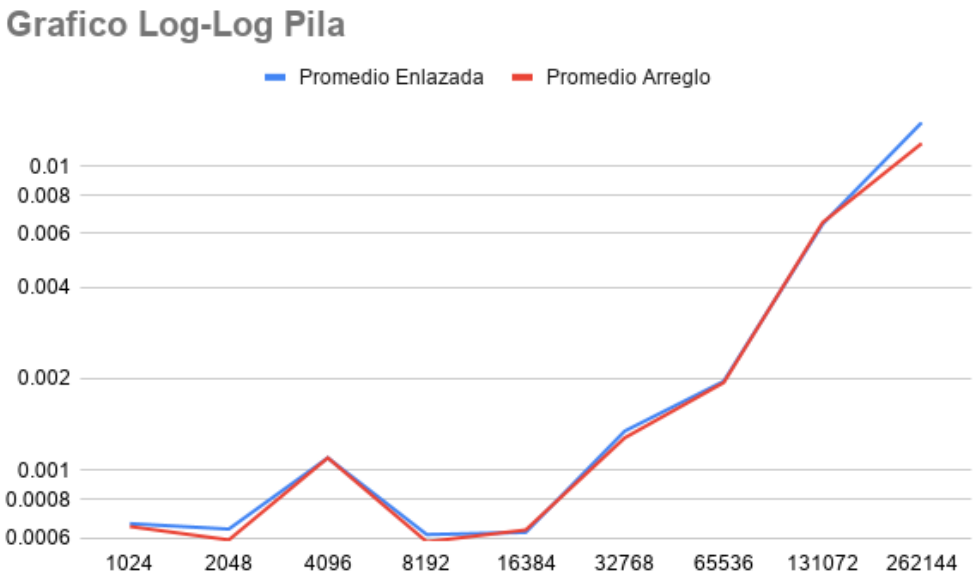


Figura 1: Gráfico Log-Log para la Pila (PilaEnlazada vs. PilaArreglo).

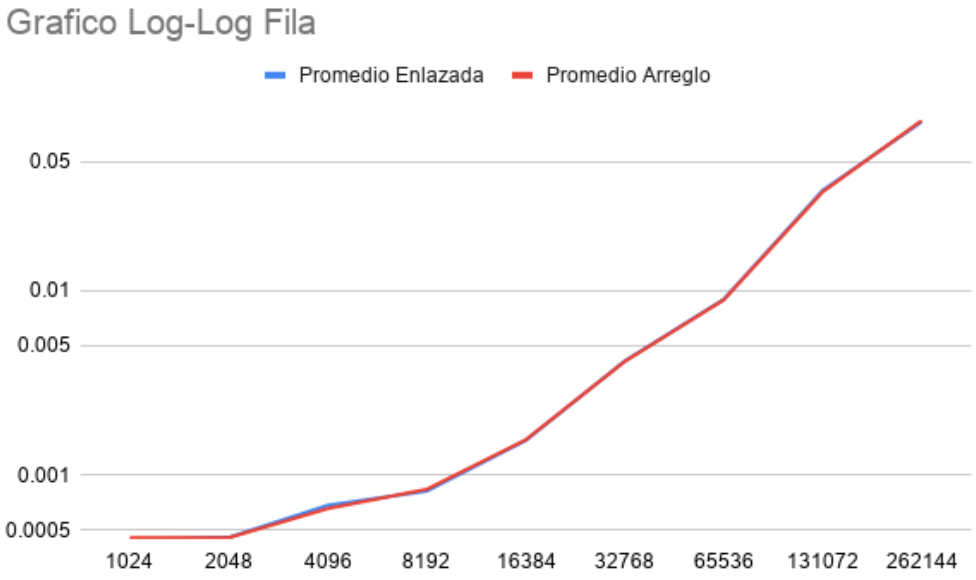


Figura 2: Gráfico Log-Log para la Fila (FilaEnlazada vs. FilaArreglo).

4.2. Factores Constantes y Rendimiento Práctico (Pregunta 4)

A pesar de que las implementaciones enlazadas y basadas en arreglos dinámicos comparten la misma complejidad asintótica amortizada $O(1)$ para sus operaciones individuales, la evidencia experimental demuestra diferencias en sus factores constantes de ejecución.

Para volúmenes masivos de datos ($N \geq 65{,}536$), las versiones con arreglo dinámico (`PilaArreglo` y `FilaArreglo`) superan en velocidad a las versiones enlazadas. Esta ventaja operacional se fundamenta en la **localidad espacial en la memoria caché** de la CPU: al almacenar elementos en contigüidad física dentro de un arreglo, el procesador aprovecha el *prefetching* y las líneas de caché $L1/L2/L3$. Por el contrario, la lista enlazada requiere la instanciación dispersa de objetos tipo `Nodo` en la memoria *Heap*, lo que incrementa sustancialmente las faltas de caché (*cache misses*) y la sobrecarga en la gestión de memoria.

Por otro lado, se observa que las operaciones sobre la Fila requieren un tiempo global mayor que sobre la Pila. Esto responde a la gestión doble de referencias (frente y final en la fila versus únicamente el tope en la pila) y a la sobrecarga añadida por las consultas de adyacencia en la estructura del grafo del sitio web.

4.3. Comportamiento del Redimensionamiento (Pregunta 5)

El análisis del mecanismo de redimensionamiento dinámico en `PilaArreglo` y `FilaArreglo` confirma la validez de la estrategia de duplicación de capacidad (capacidad $\times$ 2).

A lo largo de las pruebas de escalabilidad ($N = 2^{10}$ a 2^{18}), la cantidad de operaciones de reasignación crece de manera logarítmica ($\log_2 N$), garantizando que el costo de copiar arreglos se diluya entre todas las operaciones de inserción. Esto asegura una complejidad de tiempo amortizado de $O(1)$"por operación individual. Si se hubiera utilizado un esquema de crecimiento lineal (+1 o +K espacios), el costo total habría degenerado a una complejidad cuadrática $O(N^2)$, haciendo inviable el procesamiento para los volúmenes masivos evaluados en este experimento.

5. Conclusiones y Discusión Teórica

5.1. Análisis Teórico de Uso de Memoria (Pregunta 6)

- **Lista Enlazada:** Consume memoria proporcional exacta a la cantidad de elementos activos, pero posee un alto *overhead* por objeto/nodo (referencia al dato + puntero `siguiente`).
- **Arreglo Redimensional:** Evita el *overhead* de punteros por elemento, pero sufre de subutilización temporal de memoria (pudiendo tener hasta un 50 % de capacidad reservada no utilizada antes de encogerse).

5.2. Conclusiones Generales

1. Se ratifica empíricamente la hipótesis teórica: las implementaciones de Pila y Fila mediante arreglos dinámicos y listas enlazadas ofrecen una complejidad lineal $O(N)$ en el tiempo total de procesamiento.

2. Para sistemas de alto rendimiento y volúmenes masivos de datos, las implementaciones basadas en arreglos dinámicos son superiores debido a la localidad espacial en la memoria caché del procesador.

3. La estrategia de duplicación de capacidad es indispensable en estructuras basadas en arreglos para mantener la garantía amortizada de $O(1)$ en operaciones de inserción y evitar la degradación a un tiempo cuadrático $O(N^2)$.